System and Device Programming
On Off Exam
11.07.2024

Ex 1 (1.5 points)

Suppose to run the following program using the command:
`./pgrm 2`
Indicate how many PIDs the program displays. Note that wrong answers imply a penalty in the final score.

```
#define N 100

int main (int argc, char **argv){
  int n;
  char str[N];

  setbuf (stdout, 0);
  printf ("%d ", getpid());
  n = atoi(argv[1]);
  if (n<=0)
    return 1;
  for (int i=0; i<n; i++)
    fork();
  sprintf (str, "%d", n-1);
  execlp (argv[0], argv[0], str, NULL);
  sprintf (str, "%s %d", argv[0], n-1);
  system (str);
}
```

Choose one or more options:

1. ⬜ 7
2. ⬜ 9
3. ⬜ 11
4. ✅ 13
5. ⬜ 15
6. ⬜ 17
7. ⬜ 19

Ex 2 (1.0 points)

Suppose to run the following program. Indicate all possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
std::list<int> f (const std::list<int>& l) {
  std::list<int> rl(l.rbegin(), l.rend());
  return rl;
}

int main() {
  std::list<int> l = {10, 15, 20, 25, 30, 35, 40, 45, 50};
  std::list<int> rl = f(l);
  for (const int &e : rl) {
    std::cout << e/5 << " ";
  }
 std::cout << std::endl;
```

```
 return 0;
}
```

Choose one or more options:

1. ⬜ 0 1 2 3 4 5 6 7 8 9
2. ⬜ 10 15 20 25 30 35 40 45 50
3. ✅ 10 9 8 7 6 5 4 3 2
4. ⬜ 10 9 8 7 6 5 4 3 2 1
5. ⬜ 1 2 3 4 5 6 7 8 9 10
6. ⬜ 2 3 4 5 6 7 8 9 10
7. ⬜ 50 45 40 35 30 25 20 15 10

Ex 3 (1.5 points)

Analyze the following code snippet in C++. Indicate all possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
template<typename Container, typename Compare>
void sc (Container& container, Compare comp) {
  container.sort(comp);
}

int main() {
  std::list<std::string> list = {
    "apple", "orange", "banana", "grape", "pineapple"};
  auto dor = [](const std::string& a, const std::string& b) {
    return a > b;
  };
  sc (list, dor);
  for (const auto & str : list) {
    std::cout << str << " ";
  }
  return 0;
}
```

Choose one or more options:

1. ⬜ apple banana grape orange pineapple
2. ⬜ 5 6 5 6 9
3. ⬜ 9 6 5 6 5
4. ⬜ apple orange banana grape pineapple
5. ✅ pineapple orange grape banana apple
6. ⬜ pineapple grape banana orange apple
7. ⬜ 5 6 6 5 9
8. ⬜ 9 5 6 6 5

Ex 4 (1.5 points)

Analyze the following code snippet in C++. Indicate all possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
int main() {
  std::map<std::string, std::set<int>> myMap;
  myMap["third"].insert(6);
  myMap["third"].insert(8);
  myMap["third"].insert(7);
```

```
  myMap["second"].insert(4);
  myMap["second"].insert(5);
  myMap["first"].insert(2);
  myMap["first"].insert(1);
  myMap["first"].insert(3);
  for (const auto& pair : myMap) {
    std::cout << pair.first << ": ";
    for (const auto& num: pair.second) {
      std::cout << num << " ";
    }
  }
  std::string key = "second";
  if (myMap.find(key) != myMap.end()) {
    for (const auto& num : myMap[key]) {
      std::cout << num << " ";
    }
    std::cout << std::endl;
  } else {
    std::cout << "Key \"" << key << "\" not found." << std::endl;
  }
   return 0;
}
```

Choose one or more options:

1. ⬜ 1 2 3 4 5 6 7 8 4 5
2. ⬜ first: second: third:
3. ⬜ first: 1 2 3 second: 4 5 third: 6 7 8
4. ⬜ first: second: third: 4 5
5. ✅ first: 1 2 3 second: 4 5 third: 6 7 8 4 5
6. ⬜ first: 2 1 3 second: 4 5 third: 6 8 7 4 5
7. ⬜ third: 6 7 8 second: 4 5 first: 1 2 3 4 5
8. ⬜ first: 2 1 3 second: 4 5 third: 6 8 7 4 5

Ex 5 (1.5 points)

Analyze the following code snippet in C++. When the main is executed, indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```
using namespace std;
using std::cout;
using std::endl;

class C {
private:
  …
public:
  ...
};

void f1(C &e) { cout << "{f1}";}
void f2(C e) { cout << "{f2}";}

int main() {
  cout << "{01}"; C e1;
  cout << "{02}"; C e2;
  cout << "{03}"; C *e3 = new C[3];
```

```
  cout << "{04}"; shared_ptr<C> e4 = shared_ptr<C> (new C);
  cout << "{05}"; f1 (e1);
  cout << "{06}"; f2 (e2);
  cout << "{07}"; e3[0] = e1;
  cout << "{08}"; e3[1] = e2;
  cout << "{09}"; e3[2] = (std::move(e1));
  cout << "{10}"; delete[] e3;
  cout << "{11}"; return 0;
}
```

Solution

```
{01}[C]{02}[C]{03}[C][C][C]{04}[C]{05}{f1}{06}[CC]{f2}[D]{07}[CAO]{08}[CAO]{09}[MAO]
{10}[D][D][D]{11}[D][D][D]
```

Choose one or more options:

1. ⬜ Line number 1 includes one copy constructor.
2. ✅ Line number 4 includes one standard constructor.
3. ✅ Line number 6 includes one copy constructor.
4. ⬜ Line number 7 includes one move assignment operator.
5. ✅ Line number 7 includes one copy assignment operator.
6. ✅ Line number 9 includes one move assignment operator.
7. ⬜ Line number 9 includes one copy assignment operator.
8. ⬜ Line number 10 includes one destructor.
9. ✅ Line number 11 includes three destructors.

Ex 6 (2.0 points)

Suppose to run the following program. Indicate the possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "pthread.h"
#include "semaphore.h"

sem_t *sa, *sbc, *sd, *se;
pthread_mutex_t *me;
int na, nde;

static void *TA ();
static void *TB ();
static void *TC ();
static void *TD ();
static void *TE ();

int main (int argc, char **argv) {
  pthread_t th;

  sa = (sem_t *) malloc (sizeof(sem_t));
  sbc = (sem_t *) malloc (sizeof(sem_t));
  sd = (sem_t *) malloc (sizeof(sem_t));
  se = (sem_t *) malloc (sizeof(sem_t));
  me = (pthread_mutex_t *) malloc (sizeof (pthread_mutex_t));
  na = nde = 0;
  sem_init (sa, 0, 1);
  sem_init (sbc, 0, 0);
```

```
  sem_init (sd, 0, 0);
  sem_init (se, 0, 0);
  pthread_mutex_init (me, NULL);

  setbuf(stdout, 0);

  pthread_create (&th, NULL, TA, NULL);
  pthread_create (&th, NULL, TB, NULL);
  pthread_create (&th, NULL, TC, NULL);
  pthread_create (&th, NULL, TD, NULL);
  pthread_create (&th, NULL, TE, NULL);

  pthread_exit(0);
}

static void *TA () {
  pthread_detach (pthread_self ());

  while (1) {
    sem_wait (sa);
    na++;
    if (na==1) {
      printf ("A-");
      sem_post (sbc);
    } else {
      na = 0;
      printf ("-A\n");
      sem_post (sa);
    }

  }

  return 0;
}

static void *TB () {
  pthread_detach (pthread_self ());

  while (1) {
    sem_wait (sbc);
    printf ( "B-");
    sem_post (sd);
    sem_post (se);
  }

  return 0;
}

static void *TC () {
  pthread_detach (pthread_self ());

  while (1) {
    sem_wait (sbc);
    printf ( "C-");
    sem_post (sd);
    sem_post (se);
```

```
  }

  return 0;
}

static void *TD () {
  pthread_detach (pthread_self ());

  while (1) {
    sem_wait (sd);
    pthread_mutex_lock (me);
    printf ("D");
    nde++;
    if (nde==2) {
      nde = 0;
      sem_post (sa);
    }
    pthread_mutex_unlock (me);
  }

  return 0;
}

static void *TE () {
  pthread_detach (pthread_self ());

  while (1) {
    sem_wait (se);
    pthread_mutex_lock (me);
    printf ("E");
    nde++;
    if (nde==2) {
      nde = 0;
      sem_post (sa);
    }
    pthread_mutex_unlock (me);
  }

  return 0;
}
```

Choose one or more options:

1. ⬜ A-BC-DE-A
2. ✅ A-B-ED-A
3. ⬜ A-CB-ED-A
4. ✅ A-C-DE-A
5. ✅ A-B-DE-A
6. ⬜ A-CC-EE-A
7. ✅ A-C-ED-A
8. ⬜ A-BB-DD-A

Ex 7 (1.0 points)

Analyze the following code snippet reporting the use of a condition variable implemented in the library Pthread. Indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int sharedData = 0;
int ready = 0;

void* producer(void* arg) {
  pthread_mutex_lock(&mutex);
  sleep(1);
  sharedData = 42;
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mutex);
  return NULL;
}

void* consumer(void* arg) {
  pthread_mutex_lock(&mutex);
  while (ready == 0) {
    pthread_cond_wait(&cond, &mutex);
  }
  printf("Consumer: consumed data = %d\n", sharedData);
  pthread_mutex_unlock(&mutex);
  return NULL;
}

int main() {
  pthread_t p, c;
  pthread_create(&p, NULL, producer, NULL);
  pthread_create(&c, NULL, consumer, NULL);
  pthread_join(p, NULL);
  pthread_join(c, NULL);
  pthread_mutex_destroy(&mutex);
  pthread_cond_destroy(&cond);
  return 0;
}
```

Choose one or more options:

1. ✅ The `pthread_cond_wait` function in the consumer thread waits for the condition variable to be signaled, and, in the meantime, it releases the mutex.
2. ⬜ If `pthread_cond_signal` is called before `pthread_cond_wait`, the consumer thread will immediately proceed after calling `pthread_cond_wait`.
3. ✅ If `pthread_cond_signal` is called before `pthread_cond_wait`, the signal will be lost, and the consumer thread may wait indefinitely.
4. ✅ If `pthread_mutex_unlock(&mutex)` is not called in the producer thread after `pthread_cond_signal`, the consumer thread might be blocked indefinitely.
5. ⬜ If `pthread_mutex_unlock(&mutex)` is not called in the producer thread after `pthread_cond_signal`, the program would produce a compile-time error.
6. ⬜ The `while(ready==0)` in the consumer thread can be substituted by `if(ready==0)`.
7. ⬜ If more producers are run by the main, the condition variable synchronization does not work.
8. ⬜ If more consumers are run by the main, the condition variable synchronization works only if `pthread_cond_signal` is substituted by `pthread_condition_broadcast`.
9. ✅ The shared variable `sharedData` is protected by a mutex to avoid race conditions.

Ex 8 (1.0 points)

Indicate which of the following statements on using pointers and smart pointers in C++ are correct. Note that wrong answers imply a penalty in the final score.

Choose one or more options:

1. ✅ Smart pointers are template classes in the C++ Standard Library, that manage the lifetime of dynamically allocated objects.
2. ⬜ A weak pointer allows multiple pointers to share ownership of the same object by managing a reference count that tracks how many weak pointer instances point to the same object.
3. ⬜ A weak pointer is destroyed when its reference count drops to zero.
4. ✅ The std::weak_ptr is used to reference an object managed by a std::shared_ptr without affecting its reference count, preventing cyclic references.
5. ⬜ The sequence of two instructions: `std::shared_ptr<int> ptr1 = std::make_shared<int>(10); std::shared_ptr<int> ptr2 = ptr1;` is incorrect because the second instruction shares the ownership of `ptr1` with `ptr2`.
6. ✅ The sequence of two instructions: `auto ptr = std::make_unique<int>(10); std::unique_ptr<int> ptr2 = std::move(ptr);` create a unique pointer and transfer its ownership to another one using std::move.
7. ⬜ A weak pointer can be used directly to access the managed object safely.
8. ✅ A weak pointer can be used by adopting the lock method, which returns a shared pointer if the object is still alive. If `wptr` is a weak pointer, the following is a correct example of use: `if(auto spt = wptr.lock()) {... spt ...} else {... Object no longer exists }`

Ex 9 (1.0 points)

The following code uses a message queue to transfer data between two processes. Indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```
struct msgbuf {
  long mtype;
  // LINE 1
};

int main() {
  key_t key;
  int msgid;
  struct msgbuf message;
  // LINE 2
  msgid = msgget(key, 0666 | IPC_CREAT);
  pid_t pid = fork();
  if (pid > 0) {
    message.mtype = 1;
    strcpy(message.mtext, "Hello!");
    msgsnd(msgid, &message, sizeof(message.mtext), 0);
    wait(NULL);
    msgctl(msgid, IPC_RMID, NULL);
  } else {
    msgrcv(msgid, &message, sizeof(message.mtext), 1, 0);
    printf("Received message: %s\n", message.mtext);
  }
  return 0;
}
```

Solution

```
// LINE 1 → char text[100];
```

```
// LINE 2 → key = ftok("progfile", 65);
```

Select all that apply.

1. ⬜ The child sends a message, and the parent receives it.
2. ✅ The second field of the `msgbuf` structure, il LINE 2, must be: `char mtext[100];`
3. ⬜ The second field of the `msgbuf` structure must be: `char *mtext;`
4. ✅ In LINE 2, we must create a key with `ftok`.
5. ⬜ The function `msgctl` is used to receive the message.
6. ⬜ In a message queue, messages are always received using a FIFO order.
7. ⬜ Like in the example (with parent and child), a message queue can be used only between processes sharing the same ancestor.
8. ✅ Function `msgget` creates or opens an existing message queue. Thus, it can be called more than once, for example, by two different processes.

Ex 10 (1.0 points)

The following code snippet demonstrates the use of futures and promises in C++. Indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```
int my_f (std::future<int>& f) {
  int res = 1;
  int N = //LINE 1
  for (int i=N; i> 1; i-- )
    res *=i;
  return res;
}
int main () {
  std::promise<int> p;
  std::future<int> f = // LINE 2
  std::future<int> fu = async(std::launch::deferred,my_f,PAR3);
  p.set_value(4);
  int x = fu.get();
}
```

Solution

```
// LINE 1: f.get();
// LINE 2: p.get_future();
// PAR 3: std::ref(f)
```

Select all that apply.

1. ✅ LINE 1 must be f.get();
2. ✅ LINE 2 p.get_future();
3. ⬜ LINE 2 p.get();
4. ⬜ Parameter PAR3 must be equal to ff.
5. ✅ A `std::promise` object is used to set a value or an exception that can be retrieved by a `std::future object`.
6. ✅ A `std::future` object can retrieve the value set by a `std::promise`.
7. ✅ A std::future object blocks the calling thread until the value is available or an exception is set.
8. ✅ The thread to execute function f is run only when we get the future in line `fu.get()`:
9. ⬜ The future fu must be defined as a shared future.

Ex 11 (1.0 points)

The following code snippet demonstrates the use of file locking in C POSIX. Indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```
struct flock lock;
lock.l_type = F_WRLCK;
lock.l_whence = SEEK_SET;
lock.l_start = start * sizeof(float);
lock.l_len = count * sizeof(float);

fcntl(fd, F_SETLKW, &lock);
lseek(fd, start * sizeof(float), PAR3);
for (size_t i=0; i<count; i++) {
  write(fd, &new_value, sizeof(float));
}

lock.l_type = F_UNLCK;
fcntl(fd, F_SETLK, &lock);
```

Select all that apply.

1. ✅ File locking is a limited form of synchronization that may cause starvation and deadlock.
2. ⬜ If two processes want to use file locking to access the same file, they need to use a separate strategy to synchronize their file accesses.
3. ⬜ The third parameter `PAR3` must be `SEEK_CUR`.
4. ✅ The constant `F_WRLCk` specifies that the process wants an exclusive write lock on the file section.
5. ⬜ Locks cannot extend beyond the end of a file.
6. ✅ Multiple read locks on the same file sections are always granted.
7. ⬜ Function `lseek` is used to lock the desired file section.
8. ✅ A standard critical section strategy can lock the entire file, not a section, without locking.

Ex 12 (1.0 points)

The following code snippet demonstrates the use of file mapping in C POSIX. Indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```
char *map = mmap (NULL, filesize, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (map == MAP_FAILED) {
  ...
}
for (size_t i=0; i<filesize; i++) {
  if (islower(map[i])) {
     map[i] = toupper(map[i]); // LINE 1
  } else if (isupper(map[i])) {
     map[i] = tolower(map[i]);
  }
}
if (munmap(map, filesize) == -1) {
  ...
}
```

Select all that apply.

1. ⬜ Function `munmap` forces the contents of the mapped region to be written to the disk file.
2. ⬜ We can use the function `mmap` to map the file to memory and to resize it.
3. ⬜ With read/write, we manipulate the data directly, whereas with mmap we copy the data through the kernel.

4. ✅ The file must be opened to use function `mmap`, and `fd` is the file descriptor,
5. ✅ The code transforms all mapped characters from capital to small letters and vice-versa.
6. ⬜ To transform all mapped characters from capital to small letters and vice-versa, the program should read and write the characters using the system calls `read` and `write`.
7. ✅ LINE 1 can be rewritten as *`( map+i) = toupper(*(map+i));`
8. ✅ The first parameter of the `mmap` system call specifies the address where we want the mapped region to start; for that reason, it it often equal to NULL (or 0).